HiBench Introduction

Carson Wang
(carson.wang@intel.com)

Agenda

- Background
- Workloads
- Configurations
- Benchmark Report
- Tuning Guide

Background

WHY	• Why we need big data benchmarking systems?
WHAT	• What is HiBench?
HOW	• How to use HiBench?

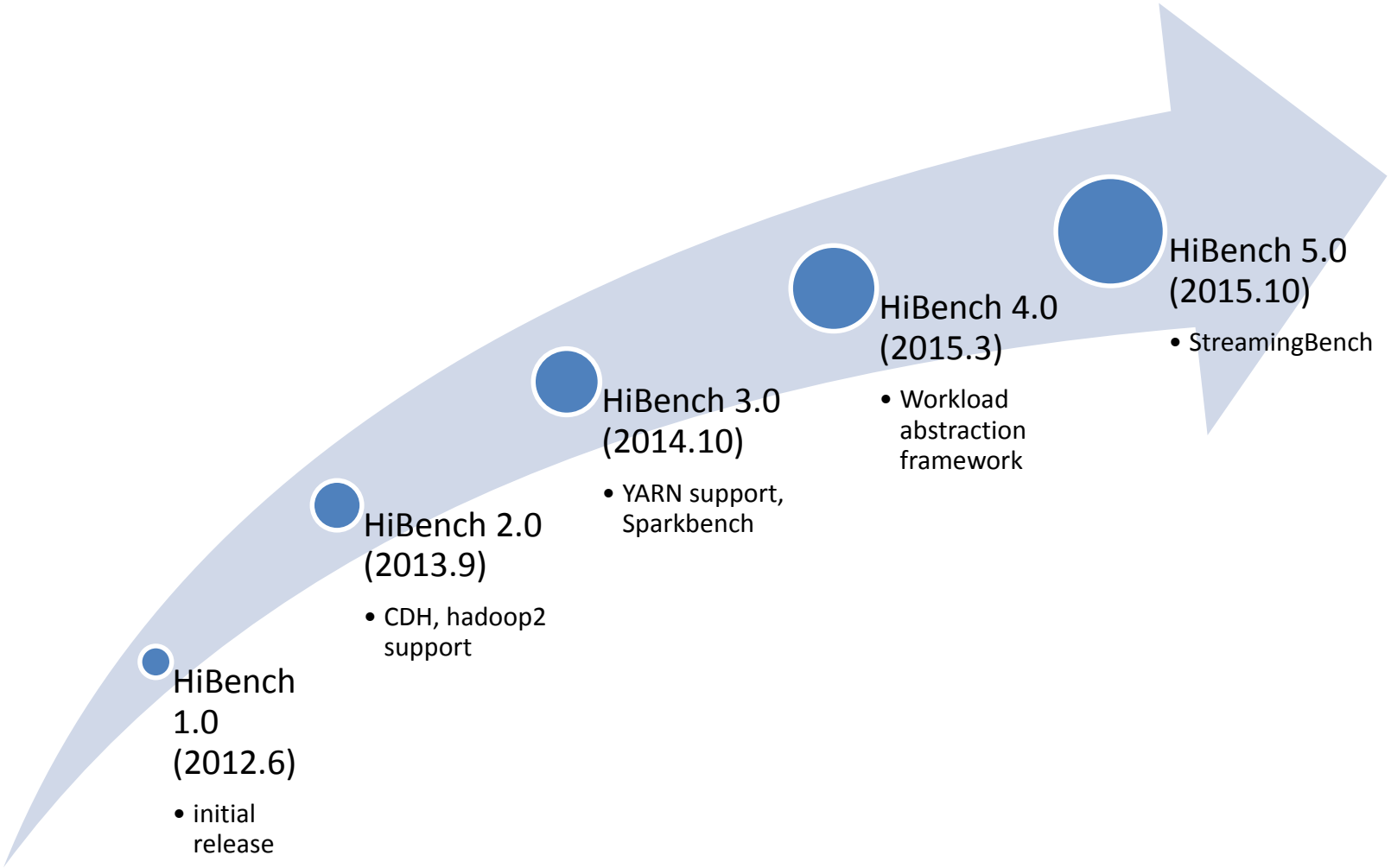
Big data ecosystem is complex

Hadoop	Spark	Streaming	Deployment	Application
MR1	Scala	Spark	Standalone	SQL
MR2	Java	Storm	YARN	MachineLearning
	Python	Storm-Trident	Kafka	Graphx
		Samza	Zookeeper	

Frequently Asked Questions from Customers

- Which framework is better?
  - Hadoop, Spark
  - Spark scala/java/python
  - SparkStreaming, Storm, or Samza
  - Standalone/YARN
- What is the performance difference between two deployments?
- How many resources are needed?
  - CPU cores, memory, network bandwidth
- Is the cluster configured properly?
  - Spark executor number, partition number tuning

HiBench Roadmap



Workload abstraction

- Micro Benchmark
- Machine Learning
- Web Search
- SQL
- Streaming
- HDFS Benchmark

Micro Benchmarks

- Sort
  - Sorts its text input data generated by RandomTextWriter.
- WordCount
  - Counts the occurrence of each word in the input data, which are generated using RandomTextWriter.
- TeraSort
  - TeraSort is a standard benchmark created by Jim Gray. Its input data is generated by Hadoop TeraGen example program.
- Sleep
  - Sleeps a few of seconds in each task to test framework scheduler.

Machine Learning

- Bayes
  - Benchmarks NaiveBayesian Classification implemented in Spark-MLLib/Mahout examples. A popular classification algorithm for knowledge discovery and data mining.
- K-Means
  - Tests the K-means (a well-known clustering algorithm for knowledge discovery and data mining) clustering in Spark-Mllib/Mahout.

Web Search

- PageRank
  - An implementation of Google's Web page ranking algorithm.
- Nutch Indexing
  - Tests the indexing sub-system in Nutch, a popular open source (Apache project) search engine.

SQL

- Scan
- Join
- Aggregation

Adapted from the Pavlo’s benchmark "A Comparison of Approaches to Large-Scale Data Analysis" and HIVE-396. It contains queries performing the typical OLAP queries described in the paper.

HDFS Benchmark

- Enhanced DFSIO
  - Tests the HDFS throughput of the Hadoop cluster by generating a large number of tasks performing writes and reads simultaneously. It measures the average I/O rate of each map task, the average throughput of each map task, and the aggregated throughput of the HDFS cluster.

Streaming

- Identity
- Sample
- Project
- Grep
- Wordcount
- Distinctcount
- Statistics

Quick Start

1. Build HiBench with Maven

mvn clean package –D spark1.5 –D MR2

2. Add basic configurations in conf/99-user_defined_properties.conf

hibench.hadoop.home	The Hadoop installation location
hibench.spark.home	The Spark installation location
hibench.hdfs.master	hdfs://sr555:8020
hibench.spark.master	yarn-client

Quick Start

3. Generate the input data and run the workload

workloads/wordcount/prepare/prepare.sh

workloads/wordcount/mapreduce/bin/run.sh

workloads/wordcount/spark/scala/bin/run.sh

workloads/wordcount/spark/java/bin/run.sh

workloads/wordcount/spark/python/bin/run.sh

4. View the report

View report/hibench.report to see the workload input data size, execution duration, throughtput(bytes/s), throughput per node.

Configuration mechanism

- Hierarchical configuration
  - Global level conf/
  - Workload level workloads/<name>/conf/
  - Framework level workloads/<name>/mapreduce/hadoop.conf
  - Language level workloads/<name>/spark/scala/scala.conf

Configuration - Parallelism

- Parallelism tuning
  - Parallelism of workload

Property name	
hibench.default.map.parallelism	Mapper numbers in MR, default parallelism in Spark
hibench.default.shuffle.parallelism	Reducer number in MR, shuffle partition number in Spark

  - Executor numbers & cores

Property name	
hibench.yarn.executors.num	Number executors in YARN mode
hibench.yarn.executors.cores	Number executor cores in YARN mode

Configuration - Memory

- Memory

Property name	
spark.executors.memory	Executor memory, standalone or YARN mode
spark.driver.memory	Driver memory, standalone or YARN mode

Configuration – Data Scale

- Data scale profile definition:
  - conf/10-data-scale-profile.conf

```
#Wordcount
hibench.wordcount.tiny.datasize          32000
hibench.wordcount.small.datasize         320000000
hibench.wordcount.large.datasize         3200000000
hibench.wordcount.huge.datasize          32000000000
hibench.wordcount.gigantic.datasize      320000000000
hibench.wordcount.bigdata.datasize       1600000000000
```

- Data scale profile selection:

```
hibench.scale.profile        large
```

Benchmark Report

- Each report/workload/framework/language has a dedicated folder which contains:
  - Console output log
  - System utilization log
  - Configurations files
    - Bash environment conf file used by workload scripts: <workload_name>.conf
    - All-in-one property based conf generated by all confs: sparkbench.conf
    - Spark/Samza/… related conf extracted from all confs: spark.conf, samza.conf, …

Report Example

- ./report
  - ./report/hibench.report
    - Summarized workload report, including workload name, execution duration, data size, throughput per cluster, throughput per node
  - ./report/<workload>/<language_api>/bench.log
    - Driver log when running this workload/api
  - ./report/<workload>/<language_api>/monitor.html
    - System utilization monitor chart on CPU, Disk, Network, Memory, Proc Load
  - ./report/<workload>/<language_api>/conf/<workload>.conf
    - Environment variable configuration generated for this workload
  - ./report/<workload>/<language_api>/conf/sparkbench/sparkbench.conf
    - Properties configuration generated for this workload
  - ./report/<workload>/<language_api>/conf/sparkbench/spark.conf
    - Spark properties configuration generated for this workload

Spark Workloads Tuning Guide

- Determine executor cores and numbers
- Determine parallelism
- Determine partition size
- Monitor system utilization

Determine Executor cores and numbers

- Set N=Number of virtual CPU cores in one node.
- Set M=Number of worker nodes in the cluster.
- Set executor cores to a number around 5:
  - Executor_cores=5
- Calculate executor number running in each node:
  - Executor_number_in_node = (N -1) / Executor_cores
- Calculate total executor numbers
  - Executor_number= Executor_number_in_node * M

Determine Parallelism

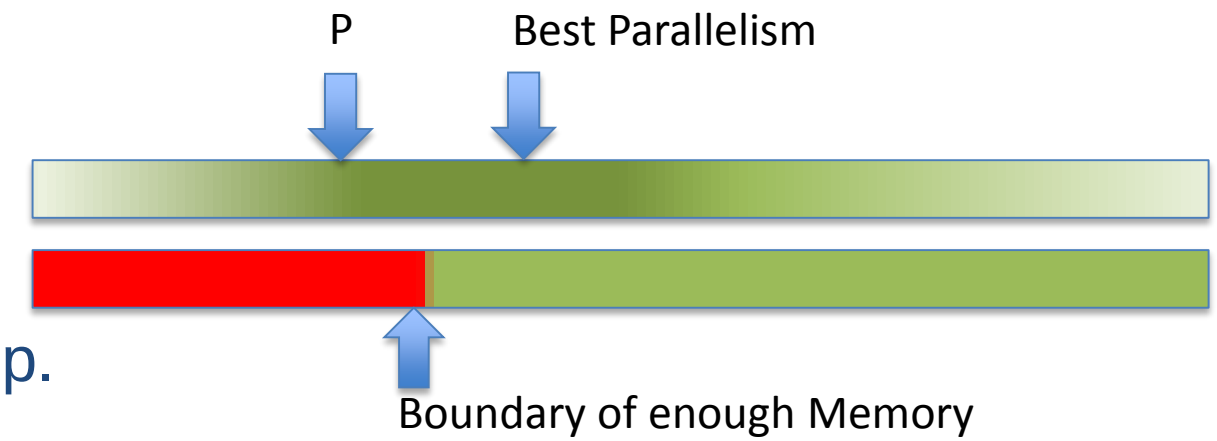
- Calculate Parallelism:

  P = Executor_cores * Executor_number

- Calculate memory for each executor:
  - Executor_memory <= Memory_node / Executor_number_in_node * 0.9

Determine partition size

- Partition size should >= P(parallelism)
  - But, a right partition size will vary with real cases.
  - Low parallelism:
    - Partition size less than P: not enough tasks for all vcores
    - Partition size too large: Tasks finished too fast. Task scheduler can't feed enough tasks.
  - Out of memory:
    - Partition size too small.
- Guideline
  - Increase partition size starting from P until performance begin to drop.



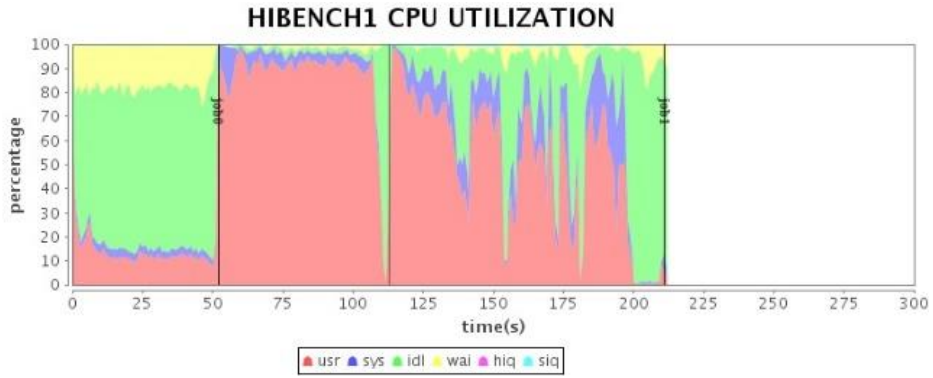
Tuning Example

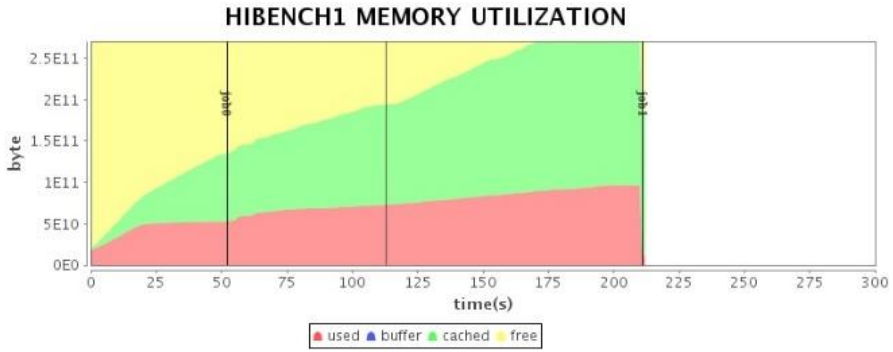
Hardware Specs

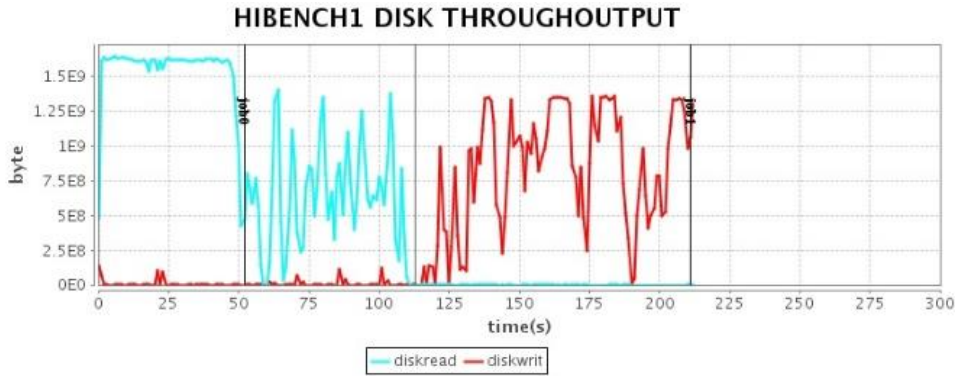
CPU	E5-2699 v3
CPU Core#	18
CPU Socket#	1
Threads Per Core#	2
Memory capacity	256 GB
Disk (SSD)	4 x 400 GB
NIC	10 Gb
Cluster Size	3 Slaves

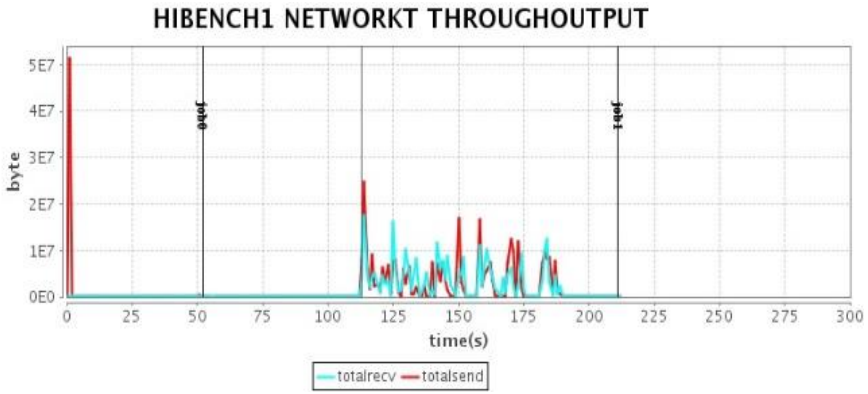
N = 36
M = 3
Executor Cores = 5
Executor Number in one node = (36 - 1) / 5 = 7
Total Executor Number = 7 * 3 = 21
Executor Memory <= 256 / 7 x 0.9 = 32 GB
Parallelism P = 5 * 21 = 105
Partition Size = 105, 210, 315, etc

Spark Tuning Example - TeraSort

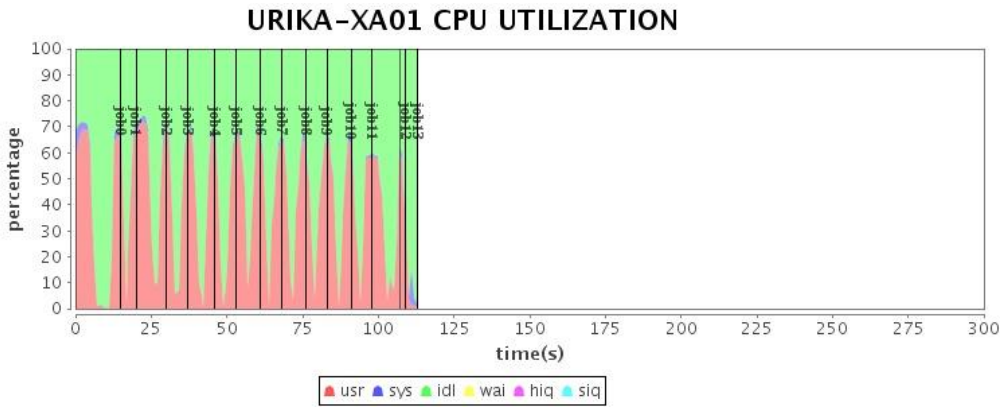


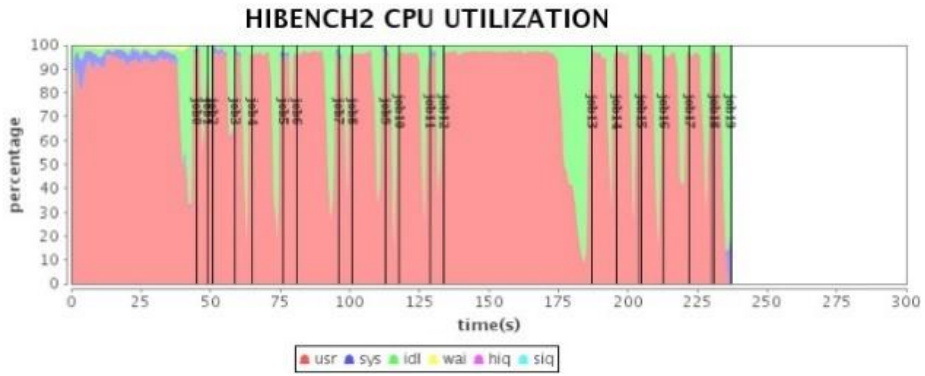






Spark Tuning Example - KMeans





Low CPU usage. Too less input Data!

RDDs produced by textFile (sequenceFile, etc) have the partitions determined by input data size and HDFS block size

HiBench 6.0 What's Next

- More real-world workloads
- More flexible build and configuration
- A better resource monitoring service

Q & A

Thanks